
Sistema de Gestión de Cine

Cómo se hizo — Informe técnico

Asignatura	Tecnologías Web
Universidad	Universidad de Granada (UGR)
Curso	2025 — 2026
Grupo / Subgrupo	A3 / 1
Autores	Carlos Manuel Pérez Molina Mario Muñoz Gutiérrez Luis Pérez Velasco

■ Repositorio del proyecto

El código fuente completo, las migraciones de base de datos, los seeders de datos de prueba y todos los recursos necesarios para revisar y ejecutar la práctica se encuentran en:

[<https://github.com/Carlitros20/sistema-cine.git>]

*El repositorio está configurado con permisos de lectura pública.
Las instrucciones de instalación se encuentran en el archivo README.md.*

Índice

1.	Introducción	3
2.	Descripción del proyecto	3
3.	Arquitectura y tecnologías	3
4.	Estructura de la base de datos	4
5.	Cumplimiento de requisitos	5
6.	Funcionalidades implementadas	6
7.	Decisiones técnicas destacadas	6
8.	Instrucciones de instalación	8
9.	Conclusiones	9

1. Introducción

Este documento describe el proceso de desarrollo del **Sistema de Gestión de Cine**. La aplicación implementa una plataforma web integral para la gestión de un cine: permite a los espectadores consultar la cartelera, comprar entradas seleccionando butacas concretas, y al personal administrador gestionar películas, salas y sesiones.

El proyecto se ha desarrollado siguiendo el guion de prácticas oficial, cubriendo los requisitos de interfaz y de funcionamiento dinámico exigidos, e incorporando mejoras adicionales orientadas a la experiencia de usuario y a la robustez del sistema.

2. Descripción del proyecto

La plataforma se estructura en torno a tres perfiles de usuario diferenciados:

Usuario no identificado

Puede navegar por la cartelera, ver el detalle de cada película (sinopsis, duración, valoraciones de otros usuarios, sesiones disponibles), filtrar el catálogo por sala, categoría o fecha, y consultar la página de contacto. No puede, sin embargo, comprar entradas ni acceder a información personal.

Usuario normal (cliente)

Tras registrarse o iniciar sesión, puede comprar entradas seleccionando butacas concretas (hasta diez por sesión), devolver sus entradas hasta cinco minutos después del inicio de la película, valorar películas mediante un sistema de cinco estrellas con comentario, marcar películas como favoritas y consultar su perfil con el historial de compras y estadísticas personales.

Usuario administrador

Tiene acceso a un panel de gestión con cuatro secciones: **Sesiones** (crear, editar y eliminar funciones), **Películas** (mantenimiento del catálogo con título, descripción, duración, categoría y póster), **Salas** (alta y baja de salas con su aforo) y **Estadísticas** (resumen de entradas vendidas, ocupación por sala y ranking de las sesiones más vendidas).

3. Arquitectura y tecnologías

La aplicación se ha construido siguiendo el patrón **Modelo-Vista-Controlador (MVC)** sobre el framework Laravel. Las tecnologías empleadas son:

Capa	Tecnología	Función
Backend	PHP 8.2 + Laravel 12	Lógica de negocio, rutas, controladores, ORM
Base de datos	MySQL 8	Persistencia de datos
Frontend	Blade + HTML5	Plantillas del lado del servidor

Estilos	CSS3 propio (sin frameworks)	Diseño visual y responsive
Interactividad	JavaScript vanilla	Bloqueo de butacas, contador, tabs
Iconografía	Bootstrap Icons (CDN)	Iconos de la interfaz
Entorno local	XAMPP + Composer	Servidor de desarrollo

4. Estructura de la base de datos

El modelo de datos consta de ocho tablas relacionadas. Se han diseñado pensando tanto en la integridad referencial como en evitar problemas de concurrencia (compra simultánea de la misma butaca por dos usuarios).

Tabla	Función
users	Usuarios del sistema con rol (customer/admin)
movies	Catálogo de películas (título, descripción, duración, categoría, póster)
rooms	Salas físicas del cine con su aforo
movie_sessions	Funciones programadas: película × sala × hora de inicio
tickets	Entradas compradas. Único por (sesión, butaca) para evitar duplicados
seat_locks	Reservas temporales de butaca (10 min) durante el proceso de compra
favorites	Tabla pivote N:M entre usuarios y películas favoritas
ratings	Valoraciones de películas (1-5 estrellas + comentario opcional)

Restricciones de integridad relevantes

Tickets: índice único sobre (movie_session_id, seat). Si dos usuarios intentan comprar simultáneamente la misma butaca, MySQL rechaza la segunda inserción a nivel de base de datos. Laravel captura la *QueryException* y muestra un mensaje claro.

Seat_locks: índice único sobre (movie_session_id, seat) y campo *expires_at* que indica cuándo expira la reserva temporal. El mismo usuario puede tener varias butacas bloqueadas a la vez para permitir la compra múltiple.

Favorites y Ratings: índice único sobre (user_id, movie_id) para evitar duplicados.

Cascadas: las relaciones usan *onDelete('cascade')*: si se elimina una película, sus sesiones, entradas y valoraciones asociadas se eliminan automáticamente.

5. Cumplimiento de requisitos

Requisitos de interfaz

Requisito	Estado
Diseño adaptable (móvil, tableta, escritorio)	✓ Cumplido
Sitio accesible (validable con herramientas W3C)	✓ Cumplido
Diseño coherente con la temática de cine	✓ Cumplido
Menú superior	✓ Cumplido
Zona central de contenido	✓ Cumplido
Menú lateral coherente	✓ Cumplido
Presentación íntegramente en CSS	✓ Cumplido
Cabecera y pie compartidos en todas las páginas	✓ Cumplido
Footer con enlace a contacto.php	✓ Cumplido
Footer con enlace a como_se_hizo.pdf	✓ Cumplido
Formularios HTML5 con campos obligatorios	✓ Cumplido
Base de datos diseñada	✓ Cumplido

Requisitos de funcionamiento dinámico

Requisito	Estado
Tres tipos de usuario diferenciados	✓ Cumplido
Sesión activa hasta cierre manual	✓ Cumplido
Nombre y tipo de usuario en esquina superior derecha	✓ Cumplido
Botón de cerrar sesión visible	✓ Cumplido
Información privada por usuario (entradas, favoritos)	✓ Cumplido
Validación de formularios	✓ Cumplido

6. Funcionalidades implementadas

Para usuarios no identificados

- Cartelera con películas en cartel (solo se muestran sesiones con más de 5 minutos hasta el inicio).
- Filtros combinables por sala, categoría y fecha.
- Detalle de cada película con sinopsis, duración, valoraciones de otros usuarios y listado de sesiones disponibles.
- Visualización del plano de butacas (en modo solo lectura).
- Página de contacto y página de cómo se hizo enlazadas desde el footer.

Para usuarios registrados

- Registro con validación estricta del formato de correo (algo@algo.algo).
- Compra de entradas con selección visual de butacas en plano de cine.
- Compra múltiple: hasta 10 butacas por sesión en una sola transacción.
- Bloqueo temporal de la butaca al seleccionarla (10 minutos), evitando que otro usuario la compre mientras decide.
- Devolución de entradas hasta 5 minutos después del inicio de la sesión.
- Valoración de películas con sistema de 5 estrellas y comentario opcional.
- Marcado de películas como favoritas con un solo clic.
- Perfil personal con estadísticas (entradas compradas, próximas sesiones, películas vistas, favoritos) y pestañas para revisar historial y favoritos.

Para administradores

- Gestión completa (CRUD) de películas, salas y sesiones.
- Panel de estadísticas con totales globales, ocupación por sala (gráfico de barras) y ranking de las 10 sesiones más vendidas con podio.
- Validación de no solapamiento al crear sesiones (no se puede programar dos funciones simultáneas en la misma sala).
- Protección de las salas con sesiones asignadas (no se pueden borrar mientras tengan sesiones).

7. Decisiones técnicas destacadas

Concurrencia en la compra de butacas

Uno de los retos principales del proyecto era garantizar que dos usuarios no pudieran comprar simultáneamente la misma butaca. Se ha implementado un sistema de doble protección:

Primer nivel — bloqueo temporal: al hacer clic sobre una butaca libre, se crea una entrada en la tabla `seat_locks` con expiración a 10 minutos. Mientras la reserva sigue vigente, los demás usuarios ven esa butaca como ocupada. Si el usuario abandona la página, un evento `beforeunload` envía una petición `sendBeacon` al servidor para liberar el bloqueo inmediatamente.

Segundo nivel — restricción de unicidad en BD: la tabla `tickets` tiene un índice único sobre (movie_session_id, seat). Aunque dos peticiones llegaran al servidor en el mismo milisegundo,

MySQL solo aceptaría la primera; la segunda lanzaría una *QueryException* que el controlador captura para mostrar un mensaje informativo.

Distribución fija de butacas e independencia del viewport

La sala de cine se renderiza siempre con 10 columnas de tamaño fijo (40 px cada una), encerrada en un contenedor con scroll horizontal. De esta forma la distribución visual es idéntica en escritorio, tableta y móvil, y no se descuadran las filas. La última fila incompleta se centra dinámicamente mediante *grid-column-start* calculado en Blade según el aforo de la sala.

Cierre de venta y devolución

La venta de entradas se cierra automáticamente 5 minutos antes del inicio de cada sesión, y la devolución se permite hasta 5 minutos después del inicio. La lógica está centralizada en el controlador para evitar duplicación.

Compras múltiples

El controlador *SeatLockController* permite a un usuario tener varios bloqueos activos simultáneamente en una misma sesión, con un máximo de 10. En la vista, las butacas seleccionadas se acumulan en un *Map* de JavaScript y se envían como *seats[]* al confirmar. El controlador *MovieController::buyTicket()* crea un ticket por cada butaca dentro de la misma transacción.

Separación de presentación

Toda la información de estilo reside en *public/css/cine.css*. Las plantillas Blade contienen únicamente estructura HTML semántica y clases CSS. Se han evitado los frameworks de estilos para tener control total sobre el diseño.

Tema visual

Se ha optado por una paleta oscura inspirada en las salas de cine reales: fondo negro con halo radial naranja en la parte superior (evoca el haz del proyector), viñeta inferior para profundidad y textura sutil de grano filmico generada mediante SVG inline. Los heroes de las secciones principales usan fotografías reales de cines (extraídas de Unsplash, banco de imágenes libre).

8. Instrucciones de instalación

Para revisar la práctica, los pasos a seguir son:

Requisitos previos

- PHP 8.2 o superior
- Composer
- MySQL 8
- (Recomendado) XAMPP como entorno integrado

Clonado del repositorio

```
git clone <URL_DEL_REPOSITORIO> cd sistema_cine
```

Instalación de dependencias

```
composer install
```

Configuración del entorno

```
cp .env.example .env php artisan key:generate
```

A continuación, editar el archivo `.env` y configurar los datos de conexión a MySQL (DB_DATABASE, DB_USERNAME, DB_PASSWORD).

Creación de la base de datos

Crear una base de datos vacía llamada *sistema_cine* desde phpMyAdmin o desde la línea de comandos de MySQL.

Migración y datos de prueba

```
php artisan migrate --seed
```

Este comando crea todas las tablas y las puebla con datos de ejemplo: dos usuarios (uno administrador y uno cliente), tres salas, varias películas con sus sesiones y algunas entradas y valoraciones de prueba.

Arranque del servidor

```
php artisan serve
```

La aplicación quedará disponible en `http://localhost:8000`.

Usuarios de prueba (creados por el seeder)

Rol	Correo	Contraseña
Administrador	admin@ciné.com	admin123
Cliente 1	cliente@ciné.com	user1234
Cliente 2	maestro@ciné.com	maestro123

9. Conclusiones

El desarrollo del Sistema de Gestión de Cine ha cubierto todos los requisitos establecidos por el guion de la práctica y ha incorporado funcionalidades adicionales que aportan robustez y mejor experiencia de usuario, como el bloqueo temporal de butacas, la compra múltiple, el sistema de valoraciones o el panel estadístico para administradores.

Durante el proceso se ha trabajado con tecnologías ampliamente utilizadas en entornos profesionales (Laravel, MySQL, arquitectura MVC) y se han aplicado buenas prácticas como la separación estricta entre HTML y CSS, la validación de datos tanto en cliente como en servidor, y el control de concurrencia a nivel de base de datos.

El resultado es una aplicación funcional, escalable y mantenible, que podría servir de base para un sistema real de gestión de un cine pequeño o mediano.

Acceso al código fuente

El repositorio del proyecto con todos los archivos (código, migraciones, seeders, hojas de estilo y recursos) está disponible en:

[<https://github.com/Carlitos20/sistema-cine.git>]